

Communication Protocol

Case A - REST API JSON

API Request | Data Encoding | Packet Capture | Python Parsing

Kelompok 02 - Sains Data Regular

HTTP :8088

4 Postman requests

64 captured packets

9

evidence screenshots

12

report pages

2 x 8

parsed DataFrame

01

Zahir Ali Izzaturahman

API Tester & Postman Collection

02

Stephanus Teo

Packet Capture & Troubleshooting

03

Enrico Lazuardi

Data Format Analysis & Python Parsing

04

Leroy Christopher Gerson

Report, PPT, GitHub & Backup Presenter

Case, tujuan praktikum, dan alur request

REST API lokal sebagai jalur komunikasi client-server yang dapat diamati dan direplikasi.

CASE A - REST API JSON

- Uji 2 GET dan 2 POST melalui Postman
- Amati HTTP request-response dan JSON
- Tangkap traffic lokal dengan Wireshark
- Parse /api/transactions ke DataFrame dan CSV

TUJUAN

Membuktikan seluruh pipeline komunikasi:

client -> protocol -> server -> format data -> analisis tabular

5

endpoint digunakan

4

Postman requests

4

HTTP exchanges

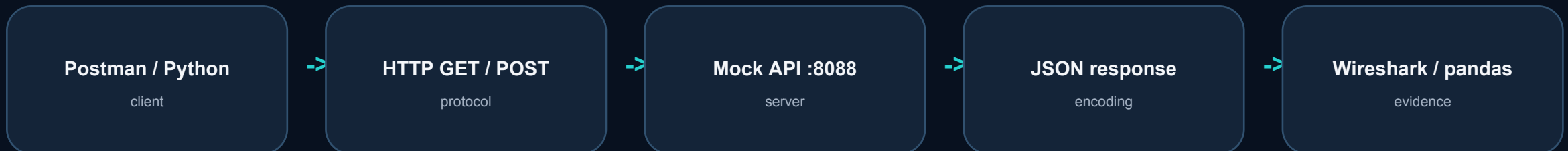
1

CSV output

BASE URL

http://127.0.0.1:8088

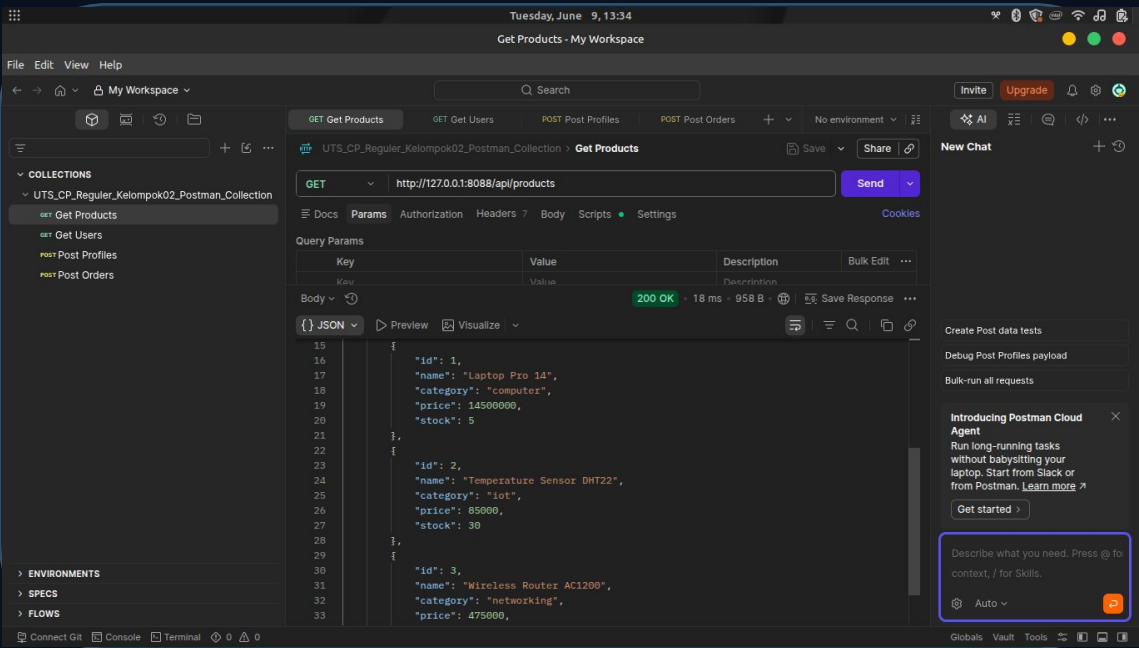
ALUR REQUEST



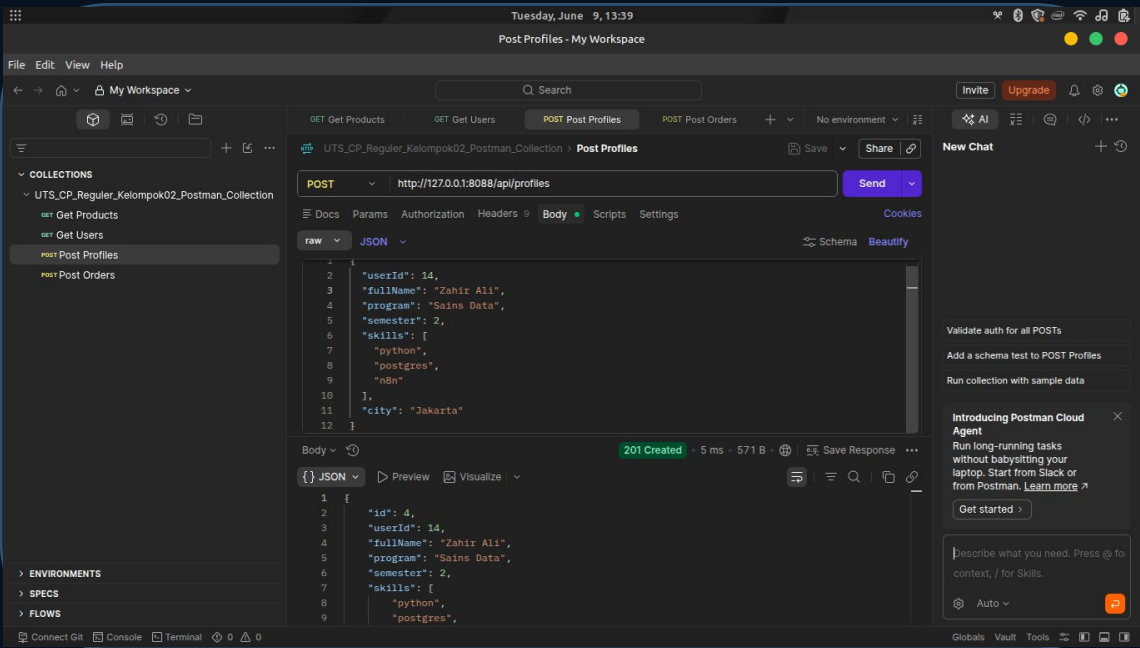
Evidence request GET/POST dan hasil response

Empat request Postman menghasilkan status sesuai semantik HTTP: GET 200, POST 201.

GET /api/products | 200 OK



POST /api/profiles | 201 Created



GET /api/products
200 OK

GET /api/users
200 OK

POST /api/profiles
201 Created

POST /api/orders
201 Created

Evidence packet capture dan analisis paket penting

PCAP diverifikasi dari adapter loopback: HTTP terlihat karena endpoint menggunakan http, bukan https.

Menangkap dari Adapter for loopback traffic capture

BerkasSuntingLihatGoTangkapAnalisisStatistikTelefoniNirkabelPerkakasBantuan

http

No.	Time	Source	Destination	Protocol	Length	Info
4	0.003840100	127.0.0.1	127.0.0.1	HTTP	257	GET /api/products HTTP/1.1
8	0.143100800	127.0.0.1	127.0.0.1	HTTP/1.0	2235	HTTP/1.0 200 OK , JSON (application/json)
17	8.635328100	127.0.0.1	127.0.0.1	HTTP	254	GET /api/users HTTP/1.1
21	8.669491300	127.0.0.1	127.0.0.1	HTTP/1.0	1586	HTTP/1.0 200 OK , JSON (application/json)
30	16.196908600	127.0.0.1	127.0.0.1	HTTP/1.1	480	POST /api/profiles HTTP/1.1 , JSON (application/json)
34	16.271413000	127.0.0.1	127.0.0.1	HTTP/1.0	268	HTTP/1.0 201 Created , JSON (application/json)
43	23.783820800	127.0.0.1	127.0.0.1	HTTP/1.1	472	POST /api/orders HTTP/1.1 , JSON (application/json)
47	23.817110000	127.0.0.1	127.0.0.1	HTTP/1.0	261	HTTP/1.0 201 Created , JSON (application/json)

> Frame 30: Packet, 480 bytes on wire (3840 bits), 480 bytes captured (3840 bits) on interface Null/Loopback

> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

> Transmission Control Protocol, Src Port: 51389, Dst Port: 8088, Seq: 1, Ack: 1, Len: 480

> Hypertext Transfer Protocol

> JavaScript Object Notation: application/json

0000 02 00 00 00 45 00 01 dc 04 c6 40 00 80 06 f6 53E....@....S

0010 7f 00 00 01 7f 00 00 01 c8 bd 1f 98 20 92 a3 cf

0020 10 3f 65 d5 50 18 00 ff 89 d4 00 00 50 4f 53 54 .?eP....POST

0030 20 2f 61 70 69 2f 70 72 6f 66 69 6c 65 73 20 48 /api/profiles H

0040 54 54 50 2f 31 2e 31 0d 0a 43 6f 6e 74 65 6e 74 TTP/1.1 Content

0050 2d 54 79 70 65 3a 20 61 70 70 6c 69 63 61 74 69 -Type: applicati

0060 6f 6e 2f 6a 73 6f 6e 0d 0a 55 73 65 72 2d 41 67 on/json User-Ag

0070 65 6e 74 3a 20 50 6f 73 74 6d 61 6e 52 75 6e 74 ent: PostmanRun

0080 69 6d 65 2f 37 2e 35 34 2e 30 0d 0a 41 63 63 65 ime/7.54 .0 Acc

0090 70 74 3a 20 2a 2f 2a 0d 0a 50 6f 73 74 6d 61 6e pt: */* Postman

00a0 2d 54 6f 6b 65 6e 3a 20 63 31 32 38 66 65 38 37 -Token: c128fe87

00b0 2d 61 39 37 36 2d 34 39 35 34 2d 61 38 34 33 2d -a976-49 54-a843-

00c0 37 63 35 39 65 39 64 61 36 32 64 65 0d 0a 48 6f 7c59e9da 62de Ho

00d0 73 74 3a 20 31 32 37 2e 30 2e 30 2e 31 3a 38 30 st: 127. 0.0.1:80

00e0 38 38 0d 0a 41 63 63 65 70 74 2d 45 6e 63 6f 64 88 Accpt-Encod

00f0 69 6e 67 3a 20 67 7a 69 70 2c 20 64 65 66 6c 61 ing: gzip, defla

0100 74 65 2c 20 62 72 0d 0a 43 6f 6e 6e 65 63 74 69 te, br Connecti

0110 6f 6e 3a 20 6b 65 65 70 2d 61 6c 69 76 65 0d 0a on: keep-alive

0120 43 6f 6e 74 65 6e 74 2d 4c 65 6e 67 74 68 3a 20 Content-Length:

0130 31 36 39 0d 0a 0d 0a 7b 0a 20 20 22 75 73 65 72 169....{ "user

Hypertext Transfer Protocol: Protocol

Paket: 60 · Ditampilkan: 8 (13.3%)

Profil: Default

64

total packets

198.48 s

capture duration

8

HTTP messages

4

request-response pairs

PAKET PENTING

- Src dan dst: 127.0.0.1

- TCP destination port: 8088

- POST Profiles frame: 480 B

- Payload JSON Content-Length: 169

- Response: HTTP/1.0 201 Created

- Content-Type: application/json

Analisis: loopback memudahkan observasi; pada HTTPS payload JSON akan dilindungi TLS.

Data format analysis: JSON, XML, dan Protobuf

Format aktual proyek adalah JSON; XML dan Protobuf dibandingkan sebagai alternatif pertukaran data.

JSON - DIPILIH

Representasi: text key-value

Kelebihan:

- mudah dibaca
- native di REST API
- langsung diparse Python

Trade-off:

- lebih verbose dari binary
- schema tidak wajib

XML

Representasi: text bertag

Kelebihan:

- struktur eksplisit
- validasi schema kuat

Trade-off:

- tag membuat payload besar
- parsing lebih kompleks

PROTOBUF

Representasi: binary

Kelebihan:

- payload ringkas
- serialisasi cepat

Trade-off:

- perlu .proto/schema
- tidak human-readable

Evidence aktual: Content-Type application/json; response berbentuk object dengan key metadata + array data.

Python parsing dan output tabel / CSV / DataFrame

requests mengambil JSON, pandas.json_normalize membentuk tabel, lalu hasil disimpan sebagai CSV.

PARSING SCRIPT

```
url = "http://127.0.0.1:8088/api/transactions"
response = requests.get(url)
response.raise_for_status()
data = response.json()

records = data["data"]
df = pd.json_normalize(records)
df.to_csv("output/parsed_result.csv", index=False)
```

Script final juga memiliki fallback untuk list, data, transactions, results, atau dict satu baris.

```
Status Code: 200
Content-Type: application/json; charset=utf-8

Raw JSON type: <class 'dict'>
Raw JSON preview:
dict_keys(['resource', 'case', 'count', 'methods', 'canonicalPath', 'itemPathExample', 'data'])

Preview DataFrame:
   id transactionId  ...   paidAt   channel
0  1  TRX-2026-001  ... 2026-06-01T09:00:00+0700  payment-gateway
1  2  TRX-2026-002  ...                None  payment-gateway

[2 rows x 8 columns]

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   id              2 non-null     int64
1   transactionId   2 non-null     object
2   orderId         2 non-null     object
3   method          2 non-null     object
4   amount          2 non-null     int64
5   status          2 non-null     object
6   paidAt          1 non-null     object
7   channel         2 non-null     object
dtypes: int64(2), object(6)
memory usage: 256.0+ bytes
None
```

200

HTTP status

2

rows

8

columns

256 B

DataFrame memory

OUTPUT

output/parsed_result.csv

Catatan Teknis, Lesson Learned, dan Kesimpulan

Hasil berhasil direplikasi; profil proyek juga menunjukkan area yang perlu diperkuat.

CATATAN TEKNIS

Capture dilakukan melalui adapter loopback

Filter http membantu memisahkan paket penting

HTTP dipakai agar request dan response dapat diamati

Parser menangani response JSON semi-terstruktur

Status code dan Content-Type berhasil divalidasi

LESSON LEARNED

HTTP menunjukkan pola request-response secara nyata

Status 200 dan 201 membuktikan operasi GET/POST berhasil

JSON mudah diparse menjadi DataFrame dan CSV

Packet capture membuktikan komunikasi pada level jaringan

Evidence terstruktur membuat hasil praktikum mudah diaudit

KESIMPULAN

Case A REST API JSON berhasil diuji end-to-end

Postman membuktikan request GET dan POST berjalan sesuai fungsi

Wireshark membuktikan traffic HTTP terjadi pada jaringan lokal

Python berhasil mengubah response JSON menjadi data tabular

Repository GitHub menyatukan seluruh evidence agar mudah direplikasi

Kesimpulan utama: seluruh evidence membuktikan REST API JSON berhasil diuji dari tiga sisi: request-response, packet capture, dan parsing data.

Kontribusi anggota, evidence map, dan demo singkat

Slide opsional: gunakan sebaga

ROLE 1 Zahir Ali Izzaturahman

Postman collection + 4 request screenshots

ROLE 2 Stephanus Teo

PCAP + 4 packet analysis screenshots

ROLE 3 Enrico Lazuardi

Python parser + CSV + DataFrame evidence

ROLE 4 Leroy Christopher Gerson

Report + PPT + repository structure

DEMO 60 DETIK

1 Postman
kirim GET/POST

->

2 Wireshark
filter http

->

3 Python
jalankan parser

->

4 CSV
cek 2 x 8 output

->

5 GitHub
tunjukkan evidence

Repository: github.com/LeroyChris/UTS-Communication-Protocol-Kelompok-02